

1. What is Data Preprocessing?

Answer:

Data preprocessing is a crucial step in data analysis and machine learning where raw data is cleaned, transformed, and organized into a structured format. It involves handling missing values, removing noise, normalizing data, and converting categorical data into numerical form to improve model accuracy and efficiency.

2. What do you mean by feature scaling or data normalization?

Answer:

Feature scaling (or data normalization) is the process of standardizing or normalizing the range of features (variables) in the dataset. Since features can have different units and scales (e.g., age vs. salary), scaling ensures that all features contribute equally to model training, preventing bias toward features with larger magnitudes.

3. Explain some techniques for feature scaling.

Answer:

Common techniques include:

- **Standardization (Z-score Normalization):** Transforms data to have a mean of 0 and a standard deviation of 1.

$$\text{Formula: } X_{new} = \frac{X - \mu}{\sigma}$$

- **Min-Max Scaling:** Scales data to a fixed range, usually [0, 1].

$$\text{Formula: } X_{new} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- **Robust Scaling:** Uses median and interquartile range (IQR) to handle outliers.

$$\text{Formula: } X_{new} = \frac{X - \text{Median}}{IQR}$$

- **Max Abs Scaling:** Scales data between [-1, 1] using the maximum absolute value.

4. What are missing values, and how do you handle them?

Answer:

Missing values are gaps in a dataset where no data is stored for certain features. Handling techniques include:

- **Deletion:** Removing rows (listwise deletion) or columns with missing values (if insignificant).
- **Imputation:**
 - **Mean/Median/Mode Imputation:** Replacing missing values with the mean (for numerical) or mode (for categorical).
 - **Forward/Backward Fill:** Used in time-series data to propagate last known values.
 - **Predictive Imputation:** Using algorithms like K-Nearest Neighbors (KNN) or regression to predict missing values.
- **Advanced Methods:** Using deep learning (autoencoders) or treating missingness as a separate category if meaningful.

1. What is PCA?

Answer:

PCA (Principal Component Analysis) is an **unsupervised dimensionality reduction technique** that transforms high-dimensional data into a lower-dimensional form while retaining most of the variance. It works by identifying **principal components**—new orthogonal axes that capture the maximum variance in the data. The first principal component explains the most variance, the second (orthogonal to the first) explains the next most, and so on.

2. Can you list out the critical assumptions of linear regression?

Answer:

The key assumptions of linear regression are:

1. **Linearity:** The relationship between independent and dependent variables should be linear.
2. **Independence:** Residuals (errors) should be independent (no autocorrelation).
3. **Homoscedasticity:** Residuals should have constant variance across all levels of predictors.
4. **Normality:** Residuals should be normally distributed (especially for small samples).
5. **No Multicollinearity:** Independent variables should not be highly correlated with each other.
6. **No Endogeneity:** Independent variables should not be correlated with the error term.

3. What is the primary difference between R-square and adjusted R-square?

Answer:

- **R-square** measures the proportion of variance in the dependent variable explained by the model, but it **always increases** with more predictors, even if they are irrelevant.
- **Adjusted R-square** adjusts for the number of predictors, **penalizing unnecessary complexity**. It only increases if a new predictor improves the model beyond chance.
- **Use Adjusted R-square** when comparing models with different numbers of predictors.

4. How is PCA used for Dimensionality Reduction?

Answer:

PCA reduces dimensionality by:

1. **Standardizing the data** (mean=0, variance=1).
 2. **Computing the covariance matrix** to understand feature relationships.
 3. **Calculating eigenvalues and eigenvectors** to identify principal components (PCs).
 4. **Selecting top-k PCs** that capture maximum variance (e.g., 95% cumulative variance).
 5. **Projecting data onto these PCs**, converting high-dimensional data into fewer uncorrelated features.
- **Result:** Retains critical information while reducing noise and redundancy.

1. What is Logistic Regression?

Answer:

Logistic Regression is a **supervised machine learning algorithm** used for **binary classification** (though it can be extended for multi-class). Unlike linear regression, which predicts continuous values, logistic regression predicts the **probability** of an instance belonging to a particular class (e.g., Yes/No, Spam/Not Spam).

It uses the **logistic (sigmoid) function** to map predicted values between **0 and 1**:

$$P(Y = 1) = \frac{1}{1 + e^{-(b_0 + b_1 X)}}$$

- If $P(Y = 1) \geq 0.5$, the output is **class 1**.
- If $P(Y = 1) < 0.5$, the output is **class 0**.

2. What are different applications of Logistic Regression?

Answer:

Logistic Regression is widely used in various domains, including:

1. **Healthcare** – Predicting disease risk (e.g., diabetes, heart attack).
2. **Finance** – Credit scoring (approving/rejecting loans).
3. **Marketing** – Customer churn prediction (will a customer leave?).
4. **Spam Detection** – Classifying emails as spam or not spam.
5. **Social Sciences** – Analyzing survey responses (e.g., voting behavior).

It is **simple, interpretable, and efficient** for linearly separable classification problems.

1. What are the basic components of an Artificial Neural Network (ANN)?

Answer:

The basic components of an ANN are:

1. **Input Layer** – Receives the initial data (features) and passes it to the network.
2. **Hidden Layers** – Intermediate layers where computation happens via weights and activation functions.
 - Each neuron applies:
$$\text{Output} = f(\text{Weighted Sum of Inputs} + \text{Bias})$$
3. **Output Layer** – Produces the final prediction (e.g., class label or continuous value).
4. **Weights & Biases** – Learned parameters that adjust during training to minimize error.
5. **Activation Function** – Introduces non-linearity (e.g., ReLU, Sigmoid).
6. **Loss Function** – Measures prediction error (e.g., MSE, Cross-Entropy).
7. **Optimizer** – Updates weights (e.g., SGD, Adam).

2. What activation function is used in the McCulloch-Pitts (M-P) Neuron Model?

Answer:

The **McCulloch-Pitts (M-P) Model** uses a **step function (threshold function)** as its activation:

$$f(x) = \begin{cases} 1 & \text{if } \sum w_i x_i \geq \theta \\ 0 & \text{otherwise} \end{cases}$$

- **Binary Output:** Fires (1) if weighted input crosses threshold (θ), else stays inactive (0).
- **Limitation:** Cannot handle non-linear data, unlike modern activation functions (e.g., Sigmoid, ReLU).

1. What is Supervised Learning?

Answer:

Supervised learning is a **machine learning paradigm** where the model is trained on **labeled data**—meaning each input example is paired with the correct output. The goal is to learn a **mapping function** from inputs to outputs so that the model can predict outcomes for new, unseen data.

Key Characteristics:

- Requires a **training dataset** with input-output pairs.
- Common tasks: **Classification** (discrete outputs) and **Regression** (continuous outputs).
- Examples: Spam detection (classification), House price prediction (regression).

2. What are Various Classification Algorithms?

Answer:

Some widely used classification algorithms include:

1. **Logistic Regression** – For binary/multi-class classification using probabilities.
2. **Decision Trees** – Splits data based on feature thresholds (interpretable).
3. **Random Forest** – Ensemble of decision trees to reduce overfitting.
4. **Support Vector Machines (SVM)** – Finds the optimal hyperplane to separate classes.
5. **k-Nearest Neighbors (k-NN)** – Classifies based on majority vote of nearest data points.
6. **Naive Bayes** – Uses Bayes' theorem with feature independence assumption.
7. **Neural Networks** – Deep learning models for complex classification tasks.

Choice depends on: Dataset size, interpretability, and problem complexity.

3. What is Prior Probability of a Class?

Answer:

The **prior probability** of a class is its **initial probability** before observing any evidence (features). It represents how likely a class is based on the training data distribution.

Formula:

$$P(\text{Class } C) = \frac{\text{Number of samples in } C}{\text{Total samples}}$$

Example:

- In a dataset with 60% "Spam" and 40% "Not Spam," the prior probabilities are:
 - $P(\text{Spam}) = 0.6$
 - $P(\text{Not Spam}) = 0.4$

Used in: Bayesian methods (e.g., Naive Bayes) to update probabilities with evidence.

1. What is a Hyperplane?

Answer:

A **hyperplane** is a decision boundary that separates data points into different classes in an n -dimensional space.

- In **2D**, it's a line (e.g., $ax + by + c = 0$).
- In **3D**, it's a plane.
- In higher dimensions, it generalizes as:

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = 0$$

where w is the weight vector and b is the bias.

Role in SVM: The optimal hyperplane maximizes the margin between classes.

2. What are the Different Kernel Functions?

Answer:

Kernels help SVM handle **non-linear data** by transforming it into a higher-dimensional space.

Common kernels include:

1. **Linear Kernel** – $K(x_i, x_j) = x_i^T x_j$
 - Works for linearly separable data.
2. **Polynomial Kernel** – $K(x_i, x_j) = (\gamma x_i^T x_j + r)^d$
 - Captures polynomial relationships (degree d).
3. **Radial Basis Function (RBF) / Gaussian Kernel** –
 $K(x_i, x_j) = e^{-\gamma \|x_i - x_j\|^2}$
 - Most popular, handles complex non-linear boundaries.
4. **Sigmoid Kernel** – $K(x_i, x_j) = \tanh(\gamma x_i^T x_j + r)$
 - Mimics neural network behavior.

Choice depends on data complexity.

3. What is the Role of Maximum Margin in SVM?

Answer:

The **maximum margin** is the **widest possible separation** between the closest data points (support vectors) of two classes.

Why it matters:

1. **Improves Generalization** – Reduces overfitting by choosing the most robust decision boundary.
2. **Better Performance** – Maximizes the model's confidence in predictions.
3. **Handles Noise** – Small perturbations in data don't easily misclassify points.

Mathematically: SVM solves the optimization problem:

$$\text{Minimize } \frac{1}{2} ||w||^2$$

$$\text{Subject to } y_i(w^T x_i + b) \geq 1$$

where $||w||$ is the margin width.

1. What are Different Clustering Techniques?

Answer:

Clustering is an **unsupervised learning** method that groups similar data points together. Major techniques include:

1. Partitioning Methods

- **K-Means:** Divides data into K clusters by minimizing variance.
- **K-Medoids:** Uses actual data points (medoids) as cluster centers (more robust to outliers).

2. Hierarchical Methods

- **Agglomerative:** Bottom-up approach (each point starts as a cluster, merges iteratively).
- **Divisive:** Top-down approach (all points start in one cluster, splits iteratively).

3. Density-Based Methods

- **DBSCAN:** Forms clusters based on dense regions (handles arbitrary shapes and noise).

4. Model-Based Methods

- **Gaussian Mixture Models (GMM):** Assumes data is generated from a mix of Gaussian distributions.

5. Grid-Based Methods

- **STING:** Divides data space into grid cells for fast clustering.

2. What is the Difference Between K-Means and K-Medoids?

Answer:

Aspect	K-Means	K-Medoids (PAM)
Center Type	Uses mean (centroid) of cluster.	Uses actual data point (medoid).
Outlier Handling	Sensitive to outliers (mean shifts).	Robust to outliers.
Cost Function	Minimizes squared Euclidean distance.	Minimizes Manhattan distance (sum of absolute differences).
Complexity	Faster ($O(n)$).	Slower ($O(n^2)$).

Example:

- In **K-Means**, adding an extreme outlier can skew the centroid.
- **K-Medoids** resists this by choosing a representative point from the data.

3. What is a Dendrogram?

Answer:

A **dendrogram** is a **tree-like diagram** used in hierarchical clustering to visualize the arrangement of clusters.

Key Features:

- **Leaves:** Represent individual data points.
- **Branches:** Show merges/splits of clusters.
- **Height:** Indicates distance between clusters (used to decide the number of clusters via a cutoff).

How to Use It?

1. **Vertical Cut:** Choose a height to determine the number of clusters.
2. **Horizontal Distance:** Shows dissimilarity between merged clusters.

Example:

- In gene expression analysis, dendrograms help group similar genes.

1. What is Agglomerative Clustering?

Answer:

Agglomerative Clustering is a **hierarchical bottom-up approach** where:

- **Each data point begins as its own cluster**
- **Iteratively merges the two most similar clusters**
- **Continues until all points form one cluster** or until stopping criteria

Key Features:

- ✓ **Builds a dendrogram** showing cluster hierarchy
- ✓ **No need to pre-specify cluster count** (unlike K-means)
- ✓ **Uses linkage methods** to determine merge order

Real-world Use:

- **Customer segmentation analysis**
- **Gene expression clustering in bioinformatics**

2. What Distance Metrics are Used?

Answer:

The algorithm requires a **distance matrix** containing all pairwise distances. Common metrics:

1. Point-to-Point Distances:

- **Euclidean:** Straight-line distance ($\sqrt{\sum (x_i - y_i)^2}$)
- **Manhattan:** Grid-like distance ($\sum |x_i - y_i|$)
- **Cosine:** Angle between vectors

2. Cluster-to-Cluster Linkage Methods:

Method	Calculation	Characteristic
Single	Minimum distance	Forms long chains
Complete	Maximum distance	Creates compact clusters
Average	Mean distance	Balanced approach
Ward's	Variance increase after merging	Minimizes total variance